Module 2: Python Programming for AI

Complete Student Study Guide & Reference Handbook

Course Module: Python Programming for AI	**Target Level:** Beginner to Applied AI Learner
Scope: Core Concepts, Data Structures, OOP & AI Libraries	**Format:** Theory + Annotated Code Implementations

1. Introduction to Python for AI

Python is a high-level, interpreted programming language known for its clear syntax and dynamic typing. In Artificial Intelligence and Data Science, Python is the primary language due to its unmatched ecosystem of specialized libraries, ease of integration, and rapid development capabilities.

> **Core Advantages of Python in AI:**
> - **Readable Syntax:** Enables programmers to express concepts in fewer lines of code than C++ or Java.
> - **Rich Library Ecosystem:** Built-in support for numerical computing (NumPy), data manipulation (Pandas), and deep learning (PyTorch, TensorFlow).
> - **Community Support:** Extensive open-source contributions and ready-to-use pre-trained model hubs.

```
# Your very first Python program
print("Hello, AI World!")
```

2. Variables and Data Types

A **variable** is a reserved memory location used to store values. Python is **dynamically typed**, meaning variable data types are inferred automatically at runtime.

Data Type	Keyword	Description	Example Code
Integer	`int`	Whole numbers without a decimal point.	`epochs = 50`
Float	`float`	Decimal and floating-point numbers.	`learning_rate = 0.001`
String	`str`	Text enclosed in single or double quotes.	`model_name = "NeuralNetwork"`
Boolean	`bool`	Logical truth values (`True` or `False`).	`is_trained = True`

```
# Variable creation and type inspection
model_name = "CNN_Classifier"  # str
epochs = 50                    # int
learning_rate = 0.001          # float
is_trained = False             # bool

# Check type dynamically
print(type(learning_rate))     # Output: <class 'float'>

# Type casting
epochs_float = float(epochs)   # Cast int to float -> 50.0
```

3. Operators and Expressions

Operators perform mathematical or logical evaluations on values (operands).

A. Arithmetic Operators

```
x = 10
y = 3

print(x + y)   # Addition: 13
print(x - y)   # Subtraction: 7
print(x * y)   # Multiplication: 30
print(x / y)   # Division: 3.3333...
print(x // y)  # Floor Division: 3
print(x % y)   # Modulus (remainder): 1
print(x ** y)  # Exponentiation (10^3): 1000
```

B. Comparison & Logical Operators

```
accuracy = 0.92
loss = 0.15

# Logic evaluation
is_optimal = (accuracy > 0.90) and (loss < 0.20)
print(is_optimal)  # Output: True
```

4. Conditional Statements

Conditional blocks (`if` , `elif` , `else`) direct program execution dynamically.

```
score = 85

if score >= 90:
    print("Grade: A - Excellent Model")
elif score >= 80:
    print("Grade: B - Good Model")
else:
    print("Grade: C - Model needs improvement")
```

5. Loops and Iteration

Loops execute a block of statements repeatedly as long as a specified condition is met.

```
# For Loop: Iterate over a sequence
for epoch in range(1, 4):
    print(f"Training Epoch {epoch} complete.")

# While Loop: Repeat until condition changes
loss = 1.0
while loss > 0.2:
    print(f"Current loss: {loss:.2f}")
    loss -= 0.3

print("Training finished!")
```

6. Functions and Exception Handling

Functions isolate reusable logic, while exception handling intercepts unexpected runtime errors.

```
def calculate_error(actual, predicted):
    '''Calculates absolute error and catches dimension mismatches.'''
    try:
        if len(actual) != len(predicted):
            raise ValueError("Input lists must have identical lengths.")
        errors = [abs(a - p) for a, p in zip(actual, predicted)]
        return sum(errors) / len(errors)
    except ValueError as err:
        print(f"Error: {err}")
        return None
    except ZeroDivisionError:
        print("Error: Input lists cannot be empty.")
        return None

# Calling function
mae = calculate_error([100, 200], [110, 190])
print("Mean Absolute Error:", mae)  # Output: 10.0
```

7. Data Structures: Lists, Tuples, Sets, Dictionaries

Structure	Ordered?	Mutable?	Duplicates?	Syntax Example
List	Yes	Yes	Allowed	`["age", "income", "score"]`
Tuple	Yes	No	Allowed	`(224, 224, 3)`
Set	No	Yes	Unique Only	`{"cat", "dog", "bird"}`
Dictionary	Yes (3.7+)	Yes	Keys Unique	`{"lr": 0.01, "batch": 32}`

```
# 1. Lists
features = ["age", "income"]
features.append("credit_score")

# 2. Tuples (Immutable setup parameters)
input_shape = (224, 224, 3)

# 3. Sets (Automatic deduplication)
classes = {"cat", "dog", "cat"}  # Output: {'cat', 'dog'}

# 4. Dictionaries (Key-Value pairs)
model_config = {
    "architecture": "CNN",
    "batch_size": 32,
    "learning_rate": 0.01
}
print(model_config["batch_size"])  # Output: 32
```

8. File Handling

Using the `with` statement guarantees clean file resource management and closure.

```
# Writing data to file
with open("metrics.txt", "w") as file:
    file.write("Epoch 1: Loss = 0.45
")
    file.write("Epoch 2: Loss = 0.31
")

# Reading data from file
with open("metrics.txt", "r") as file:
    content = file.read()
    print(content)
```

9. Object-Oriented Programming (OOP)

OOP encapsulates state (attributes) and behavior (methods) into reusable objects.

```
class NeuralNetwork:
    def __init__(self, layers, lr):
        self.layers = layers
        self.lr = lr
        self.is_trained = False

    def train(self):
        print(f"Training network with {self.layers} layers at LR={self.lr}...")
        self.is_trained = True

    def predict(self, sample):
        if not self.is_trained:
            return "Error: Model not trained!"
        return f"Prediction for {sample}: Class 1"

# Instantiate object
my_model = NeuralNetwork(layers=5, lr=0.001)
my_model.train()
print(my_model.predict([0.5, 1.2]))
```

10. Python Libraries for AI

A. NumPy

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
matrix = np.array([[1, 2], [3, 4]])

# Vectorized multiplication
scaled = arr * 2  # [2, 4, 6, 8, 10]
print("Array Mean:", np.mean(arr))
```

B. Pandas

```
import pandas as pd

data = {
    "Age": [25, 30, np.nan, 35],
    "Score": [88, 92, 95, 70]
}
df = pd.DataFrame(data)

# Preprocessing: Impute missing values with mean
df["Age"] = df["Age"].fillna(df["Age"].mean())
print(df.describe())
```

C. Matplotlib & Seaborn

```
import matplotlib.pyplot as plt
import seaborn as sns

data = [12, 15, 14, 10, 18, 22, 25, 19]
plt.figure(figsize=(5, 3))
sns.histplot(data, kde=True, color="#3182ce")
plt.title("Distribution Plot")
# plt.show()
```